



МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
имени М.В.Ломоносова



Факультет вычислительной математики и кибернетики

Компьютерный практикум по учебному курсу
«ВВЕДЕНИЕ В ЧИСЛЕННЫЕ МЕТОДЫ»

ЗАДАНИЕ № 2.

Численные методы решения дифференциальных уравнений

ОТЧЕТ

о выполненном задании

студентки 219 учебной группы факультета ВМК МГУ

Учар Айгуль Хулусиевны

гор. Москва

2023 г.

РЕШЕНИЕ ЗАДАЧИ КОШИ ДЛЯ ДИФФЕРЕНЦИАЛЬНОГО УРАВНЕНИЯ ПЕРВОГО ПОРЯДКА ИЛИ СИСТЕМЫ ДИФФЕРЕНЦИАЛЬНЫХ УРАВНЕНИЙ ПЕРВОГО ПОРЯДКА

Цель работы

Изучение и усвоение методов Рунге-Кутты второго и четвертого порядка точности, применяемых для численного решения задачи Коши, связанной с дифференциальным уравнением (или системой дифференциальных уравнений) первого порядка.

Постановка задачи

Рассмотрим обыкновенное дифференциальное уравнение первого порядка, разрешенное относительно производной и имеющее вид:

$$\frac{dy}{dx} = f(x, y), x_0 < x \quad (1)$$

с дополнительным начальным условием, заданным в точке:

$$y(x_0) = y_0 \quad (2)$$

Предполагается, что правая часть уравнения (1) функция такова, что гарантирует существование и единственность решения задачи Коши (1)-(2).

Если рассматривается не одно дифференциальное уравнение вида (1), а система обыкновенных дифференциальных уравнений первого порядка, разрешенных относительно производных неизвестных функций, то задача Коши имеет вид (на примере двух дифференциальных уравнений и дополнительных условий):

$$\begin{cases} \frac{dy_1}{dx} = f_1(x, y_1, y_2), \\ \frac{dy_2}{dx} = f_2(x, y_1, y_2), x > x_0. \end{cases} \quad (3)$$

$$y_1(x_0) = y_1^{(0)}, y_2(x_0) = y_2^{(0)}. \quad (4)$$

Предполагается, что правые части уравнений, описанных в (3), выбраны таким образом, что обеспечивается существование и единственность решения задачи Коши (3)-(4). Однако теперь речь идет о системе обыкновенных дифференциальных уравнений первого порядка, представленной в форме, где производные неизвестных функций разрешены.

Цели и задачи практической работы

- 1) Решить задачи Коши (1)-(2) и (3)-(4) наиболее известными и широко используемыми на практике методами Рунге-Кутты второго и четвертого порядка точности, аппроксимировав дифференциальную задачу соответствующей разностной схемой (на равномерной сетке); полученное конечноразностное уравнение (или уравнения в случае системы), представляющие фактически некоторую рекуррентную формулу, просчитать численно;
- 2) Найти численное решение задачи и построить его график;
- 3) Найденное численное решение сравнить с точным решением дифференциального уравнения

Описание метода решения

Воспользуемся разложением функции (1) в ряд Тейлора для построения разностной схемы интегрирования:

$$y(x_{i+1}) = y(x_i) + y'(x_i)h + y''(x_i) \frac{h^2}{2} + \dots$$

Заменяем вторую производную в этом разложении выражением

$$y''(x_i) = (y'(x_i))' = f'(x_i, y(x_i)) \approx \frac{f(\tilde{x}, \tilde{y}) - f(x_i, y(x_i))}{\Delta x}, \text{ где}$$

$$\tilde{x} = x_i + \Delta x, \tilde{y} = y(x_i + \Delta x)$$

Отметим, что Δx подбирается из условия достижения наибольшей точности записанного выражения. Для дальнейших выкладок произведем замену величины $\tilde{y}$ разложением в ряд Тейлора:

$$\tilde{y} = y(x_i + \Delta x) = y(x_i) + y'(x_i)\Delta x + \dots$$

Для исходного уравнения (1) построим вычислительную схему:

$$\begin{aligned} y_{i+1} &= y_i + f(x_i, y_i)h + \frac{h^2}{2\Delta x} (f(x_i + \Delta x, y_i + y'_i \Delta x) - f(x_i, y_i)) \\ &= y_i + h \left[\left(1 - \frac{h}{2\Delta x}\right) f(x_i, y_i) + \frac{h}{2\Delta x} f\left(x_i + \frac{\Delta x}{h}h, y_i + f(x_i, y_i) \frac{\Delta x}{h}h\right) \right] \end{aligned}$$

Положим $\alpha = \frac{h}{2\Delta x} = \frac{1}{2}$;

$$y_{i+1} = y_i + \frac{h}{2} f(x_i, y_i) + \frac{h}{2} f(x_i + h, y_i + hf(x_i, y_i)).$$

Для системы дифференциальных уравнений формула верна для всех y^j .

В ситуациях, когда точности схемы Рунге-Кутты второго порядка оказывается недостаточно, часто применяется схема Рунге-Кутты четвертого порядка точности следующего вида:

$$y_{i+1} = y_i + h \frac{k_1 + 2k_2 + 2k_3 + k_4}{6}, \text{ где } k_1 = f(x_i, y_i),$$

$$k_2 = f\left(x_i + \frac{h}{2}, y_i + \frac{h}{2}k_1\right), k_3 = f\left(x_i + \frac{h}{2}, y_i + \frac{h}{2}k_2\right), k_4 = f(x_i + h, y_i + hk_3)$$

Описание программы

- *def sec_order* (*x0*, *h*, *x1*, *y0*): функция находит, возвращает и выводит значения искомой функции задачи Коши методом Рунге-Кутты второго порядка точности $\frac{dy}{dx} = f(x, y)$, $y(x_0) = y_0$ при значениях аргумента, начиная с *x0* до *x1* с шагом *h*;
- *def fourth_order* (*x0*, *h*, *x1*, *y0*): функция находит, возвращает и выводит значения искомой функции задачи Коши методом Рунге-Кутты четвертого порядка точности $\frac{dy}{dx} = f(x, y)$, $y(x_0) = y_0$ при значениях аргумента, начиная с *x0* до *x1* с шагом *h*;
- *def system_sec_order* (*x0*, *h*, *x1*, *y10*, *y20*): функция находит, возвращает и выводит значения искомым функций при значениях аргумента, начиная с *x0* до *x1* с шагом *h*, задачи Коши для системы дифференциальных уравнений методом Рунге-Кутты второго порядка точности

$$\begin{cases} \frac{dy_1}{dx} = f_1(x, y_1, y_2), \\ \frac{dy_2}{dx} = f_2(x, y_1, y_2), \end{cases} \quad y_1(x_0) = y_{10}, \quad y_2(x_0) = y_{20};$$

- *def system_fourth_order* (*x0*, *h*, *x1*, *y10*, *y20*): функция находит, возвращает и выводит значения искомым функций при значениях аргумента, начиная с *x0* до *x1* с шагом *h*, задачи Коши для системы дифференциальных уравнений методом Рунге-Кутты четвертого порядка точности

$$\begin{cases} \frac{dy_1}{dx} = f_1(x, y_1, y_2), \\ \frac{dy_2}{dx} = f_2(x, y_1, y_2), \end{cases} \quad y_1(x_0) = y_{10}, \quad y_2(x_0) = y_{20};$$

- *def solution*(*x*): функция возвращает в точке *x* значение функции

$$y = e^{-\frac{1}{6}x^2(-3+2x)}$$

- *def system_solution1*(*y*, *x*): вспомогательная функция для библиотечной функции *odeint* поиска решений системы ОДУ
- *def f*(*x*, *y*): функция возвращает значение правой части дифференциального уравнения $y' = (x - x^2)y$;
- *def f1*(*x*, *y1*, *y2*): функция возвращает значение правой части первого дифференциального уравнения системы $y'_1 = \sin(1.4 * y_1^2) - x + y_2$
- *def f2*(*x*, *y1*, *y2*): функция возвращает значение правой части второго дифференциального уравнения системы $y'_2 = x - y_1 - 2.2y_2^2 + 1$

Исходный код

```
import numpy as np
from scipy.integrate import odeint
import math
import matplotlib.pyplot as plt

def sec_order(x0, h, x1, y0):
    n = int((x1 - x0) / h + 1)
    y = np.zeros(n)
    print('x =', x0, 'y =', y0)
    y[0] = y0

    for i in range(1, n):
        y[i] = y[i - 1] + h * f(x0, y[i - 1]) / 2 + h * f(x0 + h, y[i - 1] + h * f(x0, y[i - 1])) / 2
        x0 = x0 + h
        print('x =', x0, 'y =', y[i])

    return y

def system_sec_order(x0, h, x1, y10, y20):
    n = int((x1 - x0) / h + 1)
    y1 = np.zeros(n)
    y2 = np.zeros(n)
    print('x =', x0, 'y1 =', y10, 'y2 =', y20)

    y1[0] = y10
    y2[0] = y20

    for i in range(1, n):
        y1[i] = y1[i - 1] + h * f1(x0, y1[i - 1], y2[i - 1]) / 2 + h * f1(x0 + h, y1[i - 1]
            + h * f1(x0, y1[i - 1], y2[i - 1]), y2[i - 1] + h * f2(x0, y1[i - 1], y2[i - 1])) / 2
        y2[i] = y2[i - 1] + h * f2(x0, y1[i - 1], y2[i - 1]) / 2 + h * f2(x0 + h, y1[i - 1]
            + h * f1(x0, y1[i - 1], y2[i - 1]), y2[i - 1] + h * f2(x0, y1[i - 1], y2[i - 1])) / 2
        x0 = x0 + h
        print('x =', x0, 'y1 =', y1[i], 'y2 =', y2[i])

    return y1, y2
```

```

def fourth_order(x0, h, x1, y0):
    n = int((x1 - x0) / h + 1)
    y = np.zeros(n)
    print('x =', x0, 'y =', y0)

    y[0] = y0

    for i in range(1, n):
        k1 = f(x0, y[i - 1])
        k2 = f(x0 + h / 2, y[i - 1] + k1 * h / 2)
        k3 = f(x0 + h / 2, y[i - 1] + k2 * h / 2)
        k4 = f(x0 + h, y[i - 1] + k3 * h)
        y[i] = y[i - 1] + h * (k1 + 2 * k2 + 2 * k3 + k4) / 6
        x0 = x0 + h
        print('x =', x0, 'y =', y[i])

    return y

def system_fourth_order(x0, h, x1, y10, y20):
    n = int((x1 - x0) / h + 1)
    y1 = np.zeros(n)
    y2 = np.zeros(n)
    print('x =', x0, 'y1 =', y10, 'y2 =', y20)

    y1[0] = y10
    y2[0] = y20

    for i in range(1, n):
        k1 = f1(x0, y1[i - 1], y2[i - 1])
        m1 = f2(x0, y1[i - 1], y2[i - 1])
        k2 = f1(x0 + h / 2, y1[i - 1] + k1 * h / 2, y2[i - 1] + m1 * h / 2)
        m2 = f2(x0 + h / 2, y1[i - 1] + k1 * h / 2, y2[i - 1] + m1 * h / 2)
        k3 = f1(x0 + h / 2, y1[i - 1] + k2 * h / 2, y2[i - 1] + m2 * h / 2)
        m3 = f2(x0 + h / 2, y1[i - 1] + k2 * h / 2, y2[i - 1] + m2 * h / 2)
        k4 = f1(x0 + h, y1[i - 1] + k3 * h, y2[i - 1] + m3 * h)
        m4 = f2(x0 + h, y1[i - 1] + k3 * h, y2[i - 1] + m3 * h)
        y1[i] = y1[i - 1] + h * (k1 + 2 * k2 + 2 * k3 + k4) / 6
        y2[i] = y2[i - 1] + h * (m1 + 2 * m2 + 2 * m3 + m4) / 6

        x0 = x0 + h
        print('x =', x0, 'y1 =', y1[i], 'y2 =', y2[i])

    return y1, y2

```

```

def solution(x):
    tmp = (-1/6) * x*x * (-3+2*x)
    return np.exp(tmp)

def system_solution(y, x):
    y1, y2 = y
    return [math.sin(1.4 * y1 * y1) - x + y2, x - y1 - 2.2 * y2 * y2 + 1]

def f(x, y):
    return (x - x * x) * y

def f1(x, y1, y2):
    return math.sin(1.4 * y1 * y1) - x + y2

def f2(x, y1, y2):
    return x - y1 - 2.2 * y2 * y2 + 1

def main():
    print('Please choose a Cauchy problem you want to solve')
    print(">>y' = (x - x^2)y; y(0) = 1; (1)")
    print(">>y1' = sin(1.4 * y1^2) - x + y2; y2' = x - y1 - 2.2 * y2^2 + 1; y1(0) = 1; y2(0) = 0.5 (2)")

    mode = int(input('mode (1 or 2) = '))
    h = float(input('h (step) = '))
    x_max = float(input('x_max = '))

    n = int((x_max - 0.0) / h + 1)

    x = np.linspace(0.0, x_max, n)

    if mode == 1:
        print('Runge-Kutta method of the second order of accuracy')
        y_2 = sec_order(0, h, x_max, 1)
        print('Runge-Kutta method of the fourth order of accuracy')
        y_4 = fourth_order(0, h, x_max, 1)
        plt.plot(x, y_2, c='r', label='second order')
        plt.plot(x, y_4, c='b', label='fourth order')
        plt.plot(np.linspace(0.0, x_max, 10000), solution(np.linspace(0.0, x_max, 10000)),
                  c='y', label='real solution')
        plt.legend()
        plt.show()

    else:
        print('Runge-Kutta method of the second order of accuracy')
        y1_2, y2_2 = system_sec_order(0, h, x_max, 1, 0.5)
        print('Runge-Kutta method of the fourth order of accuracy')

        y1_4, y2_4 = system_fourth_order(0, h, x_max, 1, 0.5)

        ax, pic = plt.subplots(1, 2, figsize=(20, 10))

        pic[0].plot(x, y1_2, c='r', label='y1 second order')
        pic[0].plot(x, y1_4, c='b', label='y1 fourth order')
        pic[0].plot(np.linspace(0.0, x_max, 10000), odeint(system_solution, [0.5, 1.0],
                                                           np.linspace(0.0, x_max, 10000))[:, 0], c='y', label='y1 real solution')
        pic[0].legend()

        pic[1].plot(x, y2_2, c='r', label='y2 second order')
        pic[1].plot(x, y2_4, c='b', label='y2 fourth order')
        pic[1].plot(np.linspace(0.0, x_max, 10000), odeint(system_solution, [0.5, 1.0],
                                                           np.linspace(0.0, x_max, 10000))[:, 1], c='y', label='y2 real solution')
        pic[1].legend()
        plt.show()

if __name__ == '__main__':
    main()

```


Тесты

Таблица 1 вариант 6

$$y' = (x - x^2)y, \quad y(0) = 1;$$

Точное решение: $y = e^{-\frac{1}{6}x^2(-3+2x)}$

Рассмотрим данное уравнение на отрезке $[0; 5]$ с шагом $h = 0.2$

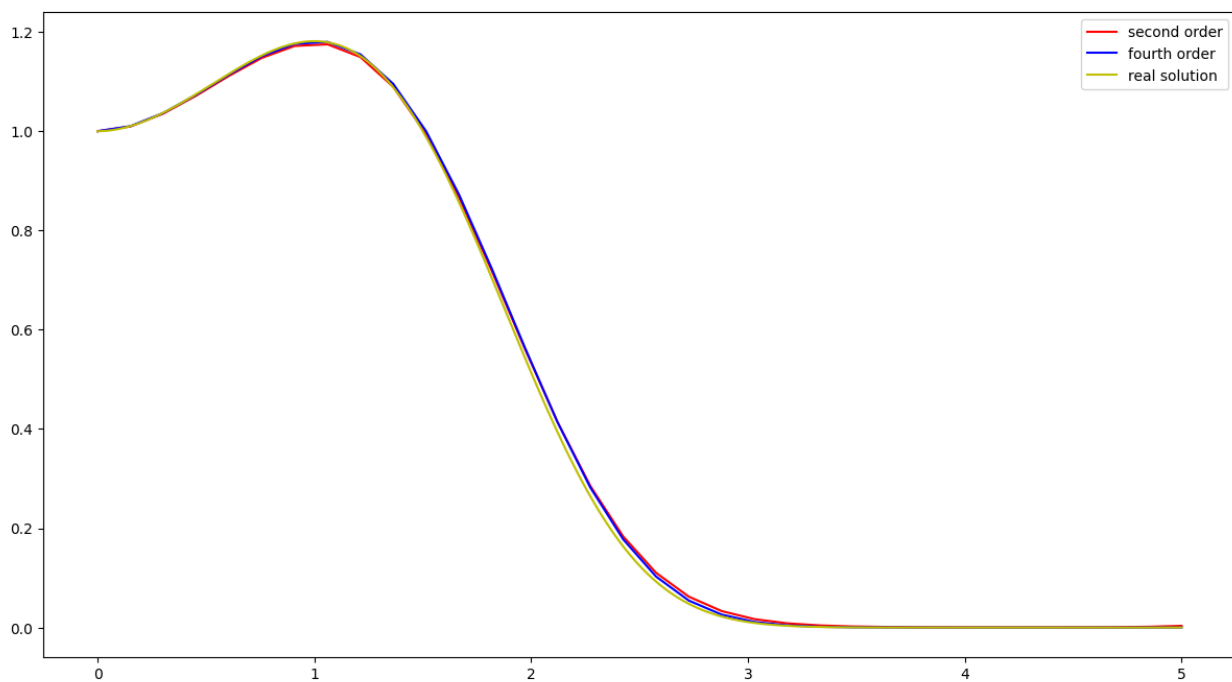


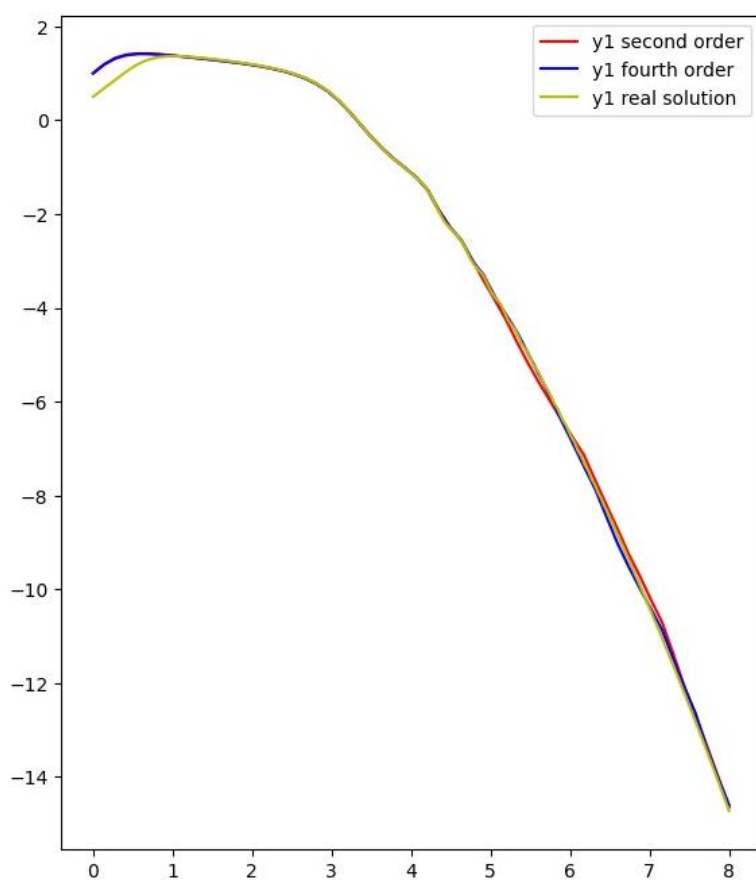
Таблица 2 вариант 17

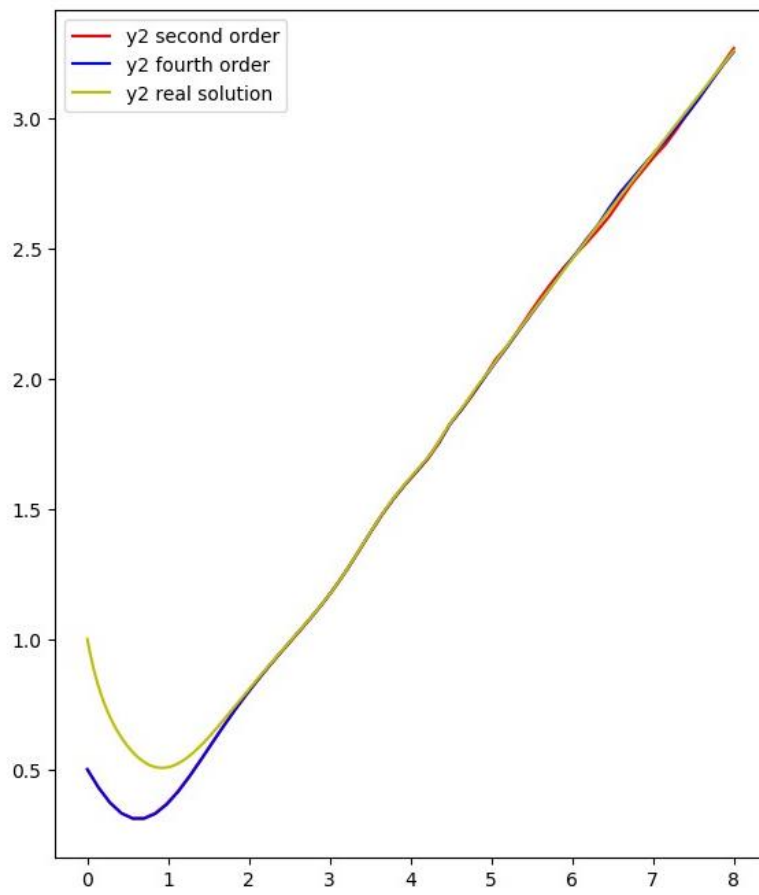
$$\begin{cases} y'_1 = \sin(1.4 * y_1^2) - x + y_2, \\ y'_2 = x - y_1 - 2.2y_2^2 + 1 \end{cases}$$

$$y_1(0) = 1, y_2(0) = 0.5$$

Рассмотрим данную систему на отрезке $[0; 8]$ с шагом $h = 0.14$

На графиках показаны графики результатов работы программы для y_1 и y_2 .





Вывод

Мы рассмотрели методы Рунге-Кутты второго и четвертого порядка точности для численного решения дифференциальных уравнений и системы дифференциальных уравнений. Важно отметить, что метод Рунге-Кутты четвертого порядка точности проявил меньшее отклонение по сравнению с методом второго порядка точности. Кроме того, при использовании меньшего шага методы Рунге-Кутты обеспечивают более эффективную сходимость.

Решение краевой задачи для обыкновенного дифференциального уравнения второго порядка, разрешенного относительно старшей производной

Цель работы

Освоить метод прогонки решения краевой задачи для дифференциального уравнения второго порядка.

Постановка задачи

Рассматривается линейное дифференциальное уравнение второго порядка вида $y'' + p(x)y' + q(x)y = -f(x), 0 < x < 1, (1)$

с дополнительными условиями в граничных точках

$$\begin{cases} \sigma_1 y(0) + \gamma_1 y'(0) = \delta_1, \\ \sigma_2 y(1) + \gamma_2 y'(1) = \delta_2. \end{cases} (2)$$

Цели и задачи практической работы

- 1) Решить краевую задачу (1)-(2) методом конечных разностей, аппроксимировав ее разностной схемой второго порядка точности (на равномерной сетке); полученную систему конечно-разностных уравнений решить методом прогонки;
- 2) Найти разностное решение задачи и построить его график;
- 3) Найденное разностное решение сравнить с точным решением дифференциального уравнения

Описание метода решения

Рассмотрим дифференциальное уравнение второго порядка с дополнительными условиями (1)-(2). Введем разностную сетку на отрезке $[a, b]: (x_0, x_1, \dots, x_n), x_i = a + ih, i = 0, \dots, n, h = \frac{b-a}{n}, n$ — число шагов сетки. Заменяем уравнение разностной сеткой, применив аппроксимацию для производных:

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} + p(x_i)\frac{y_{i+1} - y_{i-1}}{2h} + q(x_i)y_i = f(x_i),$$

$$\sigma_1 y_0 + \gamma_1 \frac{y_1 - y_0}{h} = \delta_1,$$

$$\sigma_2 y_n + \gamma_2 \frac{y_n - y_{n-1}}{h} = \delta_2$$

Введя обозначения

$$A_i = 1 - p(x_i)\frac{h}{2}, B_i = 1 + p(x_i)\frac{h}{2}, C_i = 2 - q(x_i)h^2,$$

$F_i = h^2 f(x_i)$, получим СЛАУ:

$$\begin{cases} y_0 \left(\sigma_1 - \frac{\gamma_1}{h} \right) + y_1 \frac{\gamma_1}{h} = \delta_1 \\ A_i y_{i-1} - C_i y_i + B_i y_{i+1} = F_i, i = 1, \dots, n-1 \\ y_n \left(\sigma_2 + \frac{\gamma_2}{h} \right) - y_{n-1} \frac{\gamma_2}{h} = \delta_2 \end{cases}$$

Решая СЛАУ методом прогонки: $y_i = \xi_{i+1} y_{i+1} + \eta_{i+1}, i = 0, \dots, n-1$,

получаем:

$$y_0 = \frac{\delta_1 - y_1 \frac{\gamma_1}{h}}{\sigma_1 - \frac{\gamma_1}{h}}. \text{ Тогда } \xi_1 = -\frac{\gamma_1}{h \left(\sigma_1 - \frac{\gamma_1}{h} \right)}, \eta_1 = \frac{\delta_1}{\sigma_1 - \frac{\gamma_1}{h}}.$$

Из прямого хода метода прогонки с трехдиагональной матрицей имеем:

$$\xi_{i+1} = \frac{B_i}{C_i - \xi_i A_i}, \eta_{i+1} = \frac{A_i \eta_i - F_i}{C_i - \xi_i A_i}, i = 1, \dots, n-1.$$

$$y_{n-1} = \xi_n y_n + \eta_n, y_n = \frac{\delta_2 + \frac{\gamma_2}{h} \eta_n}{\left(\sigma_2 + \frac{\gamma_2}{h} \right) \xi_n - \frac{\gamma_2}{h}}$$

Обратный ход метода прогонки: $y_i = \xi_{i+1} y_{i+1} + \eta_{i+1}, i = n-1, \dots, 0$.

Описание программы

- *def p0(x) (q0, f0, p1, q1, f1, p2, q2, f2)*: функция возвращает значение соответствующего коэффициента, зависящего от x , в уравнении $y'' + p(x)y' + q(x)y = f(x)$;
- *def solution1(x)*: функция возвращает значение в точке функции, являющейся решением краевой задачи (mode = 2) $y'' - 2y' - 3y = e^{4x}$, $0 < x < 1$, $-y(0) + y'(0) = 0.6$, $y(1) + y'(1) = 4e^3 + e^4$;
- *def solution2(x)*: функция возвращает значение в точке функции, являющейся решением краевой задачи (mode = 3) $y'' + y' = 1$, $0 < x < 1$, $y'(0) = 0$, $y(1) = 1$;
- *def test(n, x0, xn, sig1, sig2, gam1, gam2, del1, del2, p, q, f)*: функция находит и возвращает n значений решения краевой задачи вида $y'' + p(x)y' + q(x)y = f(x)$, $x_0 < x < x_n$

$$\begin{cases} \sigma_1 y(x_0) + \gamma_1 y'(x_0) = \delta_1, \\ \sigma_2 y(x_n) + \gamma_2 y'(x_n) = \delta_2, \end{cases}$$

соответствующих разбиению отрезку $[x_0, x_n]$ на $n+1$ частей

Исходный код

```
import numpy as np
import matplotlib.pyplot as plt
import math

def test(n, x0, xn, sigma1, sigma2, gamma1, gamma2, delta1, delta2, p, q, f):
    x = np.linspace(x0, xn, n + 1)
    y = np.zeros(n + 1)
    h = (xn - x0) / n
    A = np.zeros(n + 1)
    A[n] = - gamma2 * h
    B = np.zeros(n + 1)
    B[0] = gamma1 * h
    C = np.zeros(n + 1)
    C[0] = - sigma1 * h * h + gamma1 * h
    C[n] = - (sigma2 * h * h + gamma2 * h)
    F = np.zeros(n + 1)
    F[0] = delta1 * h * h
    F[n] = delta2 * h * h

    for i in range(1, n):
        A[i] = 1 - p(x[i]) * h / 2
        B[i] = 1 + p(x[i]) * h / 2
        C[i] = 2 - q(x[i]) * h * h
        F[i] = h * h * f(x[i])

    xi = np.zeros(n)
    xi[0] = B[0] / C[0]
    nu = np.zeros(n)
    nu[0] = - F[0] / C[0]

    for i in range(1, n):
        xi[i] = B[i] / (C[i] - xi[i - 1] * A[i])
        nu[i] = (A[i] * nu[i - 1] - F[i]) / (C[i] - xi[i - 1] * A[i])

    y[n] = (F[n] - A[n] * nu[n - 1]) / (A[n] * xi[n - 1] - C[n])

    for i in range(n - 1, -1, -1):
        y[i] = xi[i] * y[i + 1] + nu[i]

    return y
```



```

def p0(x):
    return -0.5

def q0(x):
    return 3

def f0(x):
    return 2*x*x

def solution1(x):
    return np.exp(-x) + np.exp(3 * x) + 0.2 * np.exp(4 * x)

def p1(x):
    return 2

def q1(x):
    return -1 / x

def f1(x):
    return 3

def solution2(x):
    return np.exp(-x) - np.exp(-1) + x

def p2(x):
    return 1.0

def q2(x):
    return 0.0

def f2(x):
    return 1.0

def main():
    print('Which problem do you want to solve?')
    print("(1)  $y'' - 0.5y' + 3y = 2x^2$ ,  $y(1)-2y'(1) = 0.6$ ,  $y(1.3) = 1$ ")
    print("(2)  $y''-2y'-3y=e^{(4x)}$ ,  $-y(0.0)+y'(0.0)=0.6$ ,  $y(1)+y'(1)=4e^3+e^4$ ")
    print("(3)  $y''+y'=1.0$ ,  $y'(0.0)=0.0$ ,  $y(1)=1.0$ ")

    mode = int(input('problem number: '))
    n = int(input('n = '))

```

```

if mode == 1:
    y = test(n, 1.0, 1.3, 1.0, 1.0, -2.0, 0.0, 0.6, 1.0, p0, q0, f0)
    x = np.linspace(1.0, 1.3, n + 1)

    plt.plot(x, y, c='r', label='y(x)')

elif mode == 2:
    y = test(n, 0.0, 1.0, -1.0, 1.0, 1.0, 1.0, 0.6, 4 * math.exp(3) + math.exp(4), p1, q1, f1)
    x = np.linspace(0.0, 1.0, n + 1)

    plt.plot(x, y, c='r', label='y(x)')
    plt.plot(x, solution1(x), c='b', label='real solution')

elif mode == 3:
    y = test(n, 0.0, 1.0, 0.0, 1.0, 1.0, 0.0, 0.0, 1.0, p2, q2, f2)
    x = np.linspace(0.0, 1.0, n + 1)

    plt.plot(x, y, c='r', label='y(x)')
    plt.plot(x, solution2(x), c='b', label='real solution')

plt.legend()
plt.show()

if __name__ == '__main__':
    main()

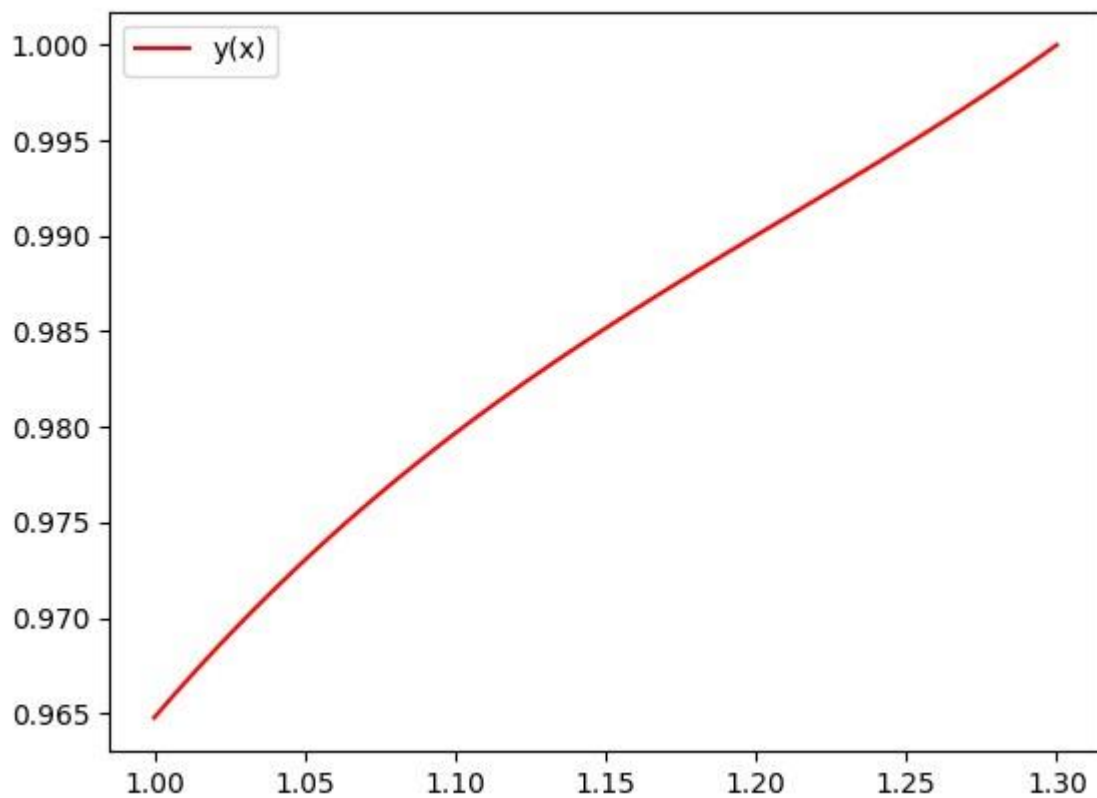
```

Тесты

Вариант 9

$$y'' - \frac{y'}{2} \cdot 3y = 2x^2; \quad y'(1) - 2y'(1) = 0.6; \quad y(1.3) = 1.$$

Рассмотрим разбиение данного отрезка $[1.0; 1.3]$ на $n = 100$ частей, получим графическое представление приближенного решения краевой задачи.

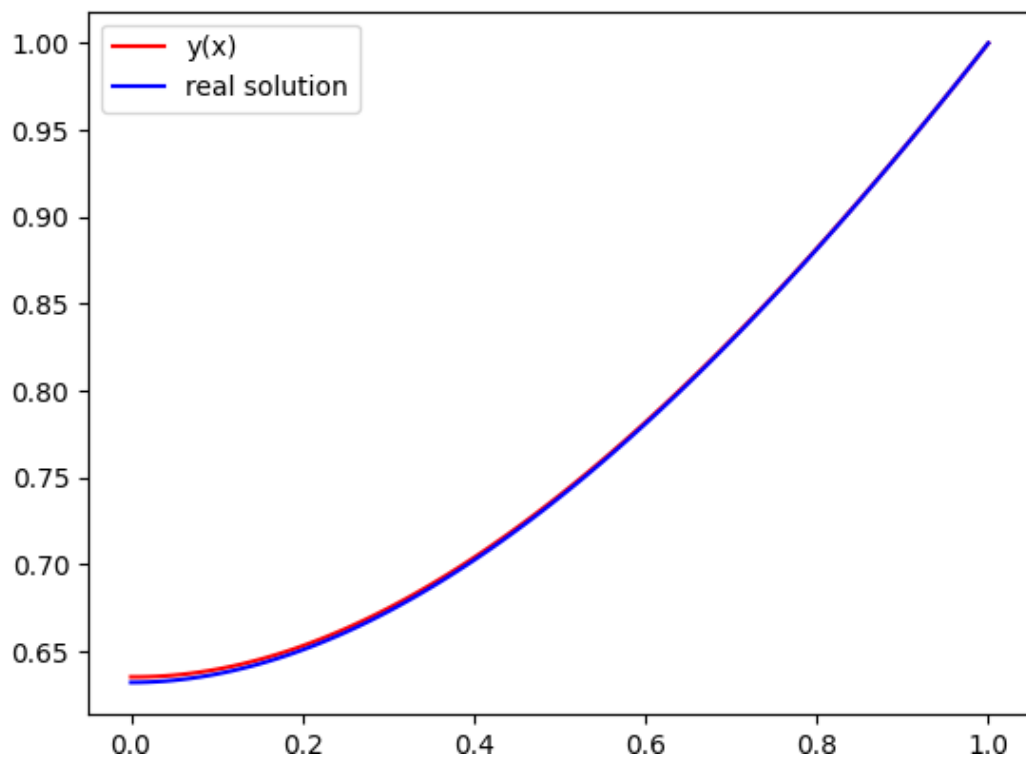


Дополнительный тест 1

$$y'' + y' = 1; y'(0) = 0; y(1) = 1$$

Заданной краевой задаче соответствует аналитическое решение вида $y = e^{-x} - e^{-1} + x$.

Рассмотрим разбиение данного отрезка $[0.0; 1.0]$ на $n = 100$ частей, получим графическое представление приближенного решения краевой задачи.

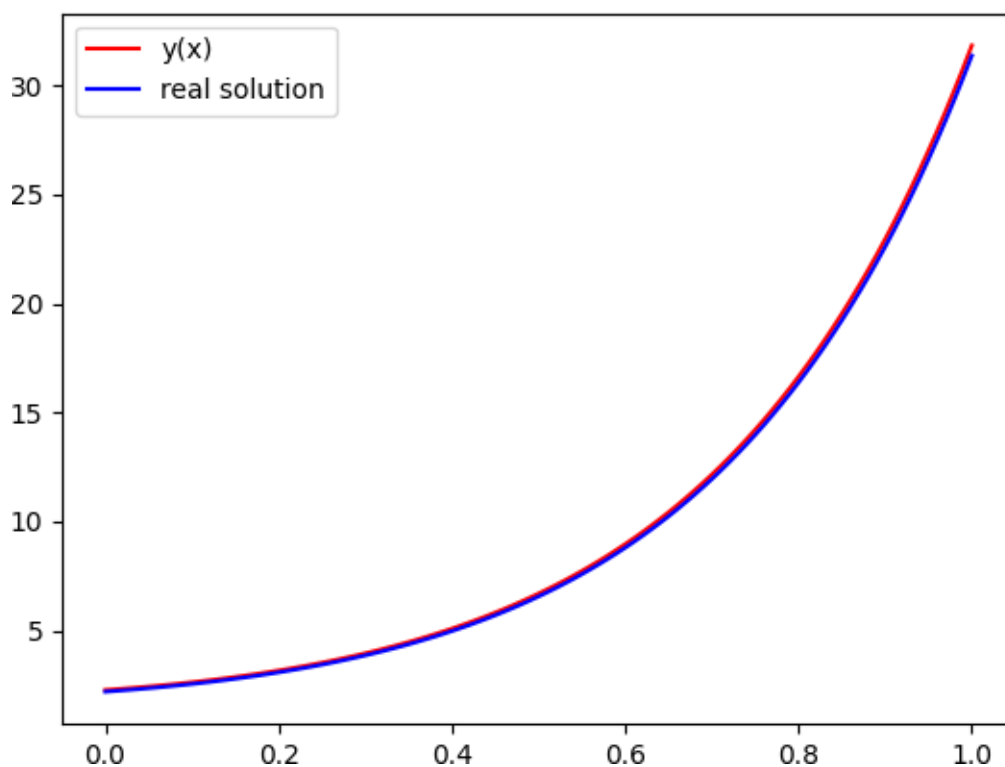


Дополнительный тест 2

$$y'' - 2y' - 3y = e^{4x}; -y(0) + y'(0) = 0.6; y(1) + y'(1) = 4e^3 + e^4$$

Заданной краевой задаче соответствует аналитическое решение вида $y = e^{-x} + e^{3x} + 0.2e^{4x}$.

Рассмотрим разбиение данного отрезка $[0.0; 1.0]$ на $n = 100$ частей, получим графическое представление приближенного решения краевой задачи.



Вывод

В ходе выполнения отчетной работы был рассмотрен метод прогонки решения краевой задачи для дифференциального уравнения второго порядка. Важно отметить, что данный метод обладает более точной сходимостью к реальному решению при увеличении числа разбиений рассматриваемого отрезка.